SCIKIT-LEARN BASICS

TUTORIAL 4

(Implementing a typical model-training workflow)

MARTIN MURIITHI

This document contains Hyperlinks



[https://numpy.org]



[https://matplotlib.org]



[https://seaborn.pydata.org]



[https://scikit-learn.org/stable/]



[https://pandas.pydata.org/docs/]

Official respective documentation linked above



Objectives

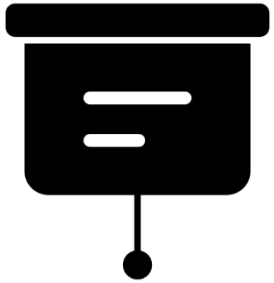
1. Data wrangling with pandas
2. Exploratory Data Analysis (EDA)
3. sklearn basics



Resources...

Hyper-link included below

1. Please note that all the resources for this entire tutorial (code reference in Jupyter Notebook, dataset, slides, question, saved model state and deployment files) are included on my GitHub repository [linked here]
2. If possible, please refer to these slides and the Jupyter Notebook side by side



Introduction

ℹ Some important points to consider before we begin coding

In this tutorial lesson…

We shall combine everything we've learned so far on pandas, matplotlib, and numpy and work towards building an end to end ML training workflow.

1. We shall perform data wrangling using pandas
2. We shall then perform Exploratory Data Analysis using pandas, numpy and matplotlib to help us get more insight into the data and select features
3. We shall then split data into training and test sets and perform data pre-processing. We'll see how to fit the pre-processing step into a scikit-learn pipeline

⚠ **Starting from STEP 3, we shall introduce the sklearn library to train and evaluate a classification ML model**

Not so fast… Let's do a refresher on Machine Learning (ML) first, shall we?

What is ML?

ML is a subset of Artificial Intelligence (AI) that uses algorithms to analyze data, identify patterns and make predictions

- ✓ In ML, computers learn from data, on their own, without direct instruction
- ✓ Prediction is improved as more data is collected and analyzed, much like how humans learn from experience

ML is the foundation on which AI is built







Let's assume we're to hard code a program that distinguishes between photos of cats and dogs. We'd have to identify features specific to cats and dogs (think about all the different features in at least 340 dog breeds and 75 cat breeds). Then, we'd have to write an algorithm to decipher these features and return the result as a dog or cat. Consider the thousands of complex rules, such as ear shape, ear position, fur, lighting, angles, variations, e.t.c. These would be nearly impossible to define explicitly

The wrong approach





✓ The Machine Learning approach makes more sense. Instead of hard coding thousands of complex visual rules, an ML model learns patterns directly from examples of cats and dog images (by itself, without direct instructions). It's a more accurate, scalable approach that's adaptable to new images

ML is hence ideal for scenarios with constantly changing data or tasks, where hard coding solutions would be impractical

The fundamental idea of ML…

Is to use data from past observations to predict unknown outcomes. For example:



An ice cream store might use an app that combines historical sales and weather records to predict how many ice creams they're likely to sell on a given day, based on the weather forecast



A doctor might use clinical data from past patients to run automated tests that predict whether a new patient is at risk from diabetes based on factors like weight, blood glucose level, and other measurements



A researcher in the Antarctic might use past observations to automate the identification of different penguin species (such as Adelie, Gentoo, or Chinstrap) based on measurements of a bird's flippers, bill, and other physical attributes.

ML is based on Math & Statistics…

Hence, it's common to think about ML models in mathematical terms.



Fundamentally, a machine learning model is a software application that encapsulates a **function, f** to calculate an output value, y, based on one or more input values, x. In mathematical notation **y = f(x)**, where y is the **label**, and x the **features**



The process of defining that function is known as **training**.

After the function has been defined, you can use it to predict new values in a process called **inferencing**.



Considering our prev. example…

To clarify the difference between **features** and **label**, let's look at it this way:



In the Ice Cream shop example, the aim is to train a model that predicts ice cream sales using weather data like temperature, rainfall, and windspeed as ***features (x)***, with sales numbers as the ***label (y)***.



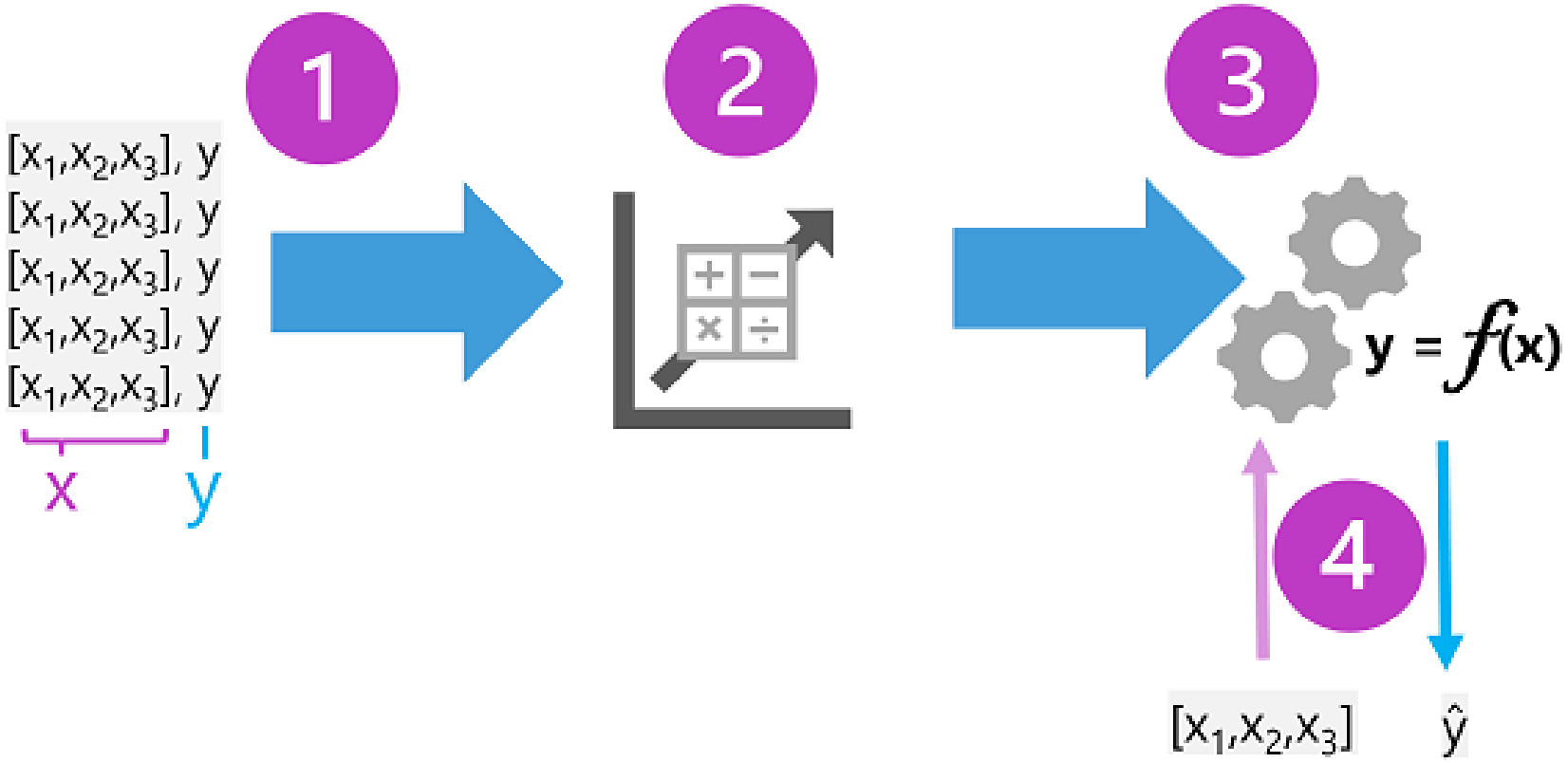
In the medical example, the aim is to predict a patient's diabetes risk using clinical data like weight, age and blood glucose. These measurements are the ***features (x)***, and the diabetes risk (1 for at risk, 0 for not) is the ***label (y)***.



In the Antarctic research example, penguin species are predicted using physical traits like flipper length and bill width as ***features (x)***, while the species - Adelie (*0*), Gentoo (*1*), or Chinstrap (*2*) serve as the ***label (y)***.

Source: Microsoft Learn

Steps involved in training & inferencing…



1 Training data includes previous observations with their **features, x** and the known **label, y,** which is the value the model aims to predict. In math, features are often called *x* and the label *y*. Since observations have multiple features, *x* is usually a ***vector*** like *[x1, x2, x3, ...]*

2

An **algorithm** analyzes data to find a relationship between features, **x** and the label, **y**, creating a function that uses ***x*** to predict ***y***. The choice of algorithm depends on the predictive problem, but the goal is to *fit* the data to calculate the label from the features.

3

The result of the algorithm is a ***model*** that encapsulates the calculation derived by the algorithm as a function - let's call it **f**. In mathematical notation: $\mathbf{y = f(x)}$

4

Now that the training phase is complete, the trained model can be used for **inferencing**. Inferencing involves inputting a set of feature values, **x**, and receiving as an output, a prediction of the corresponding label $\hat{\mathbf{y}}$ (verbalized as "y-hat")

Reference article linked [here]

Types of Machine Learning

IBM explains that ML algorithms fall into five broad categories, namely

1. Supervised ML

A type of ML where the model is trained on a *labeled dataset* ie, the training data has both features, **x**, & labels, **y**. Under Supervised ML:

(a) Regression

A form of supervised ML in which the label predicted by the model is a **numeric value.** For example, stock price, sale quantities, fuel efficiency, crop produce

(b) Classification

A form of supervised ML in which the label represents a **categorization, or *class*.** There are two common classification scenarios.

(i) Binary Classification

Binary classification models predict one of **two** mutually exclusive outcomes, e.g whether a patient is at the risk of diabetes or not, whether a student will fail or pass their final exam, gender (male or female), e.t.c

(ii) Multi-class Classification

These models predict a label that represents one of **multiple possible classes**; e.g. penguin species (*Adelie*, *Gentoo*, or *Chinstrap*), type of pet (dog, cat, or parrot)

This will be our focus for this tutorial going forward: binary classification

2. UnSupervised ML

Unsupervised machine learning involves training models using data that consists only of *feature* values, x without any known labels. The most common type of unsupervised ML is **clustering**

3. Self-supervised ML

Self-supervised learning (SSL) enables models to train themselves on unlabeled data, instead of requiring massive annotated and/or labeled datasets. SSL algorithms learn one part of the input from another part, automatically generating labels and transforming unsupervised problems into supervised ones

4. Reinforcement Learning (RL)

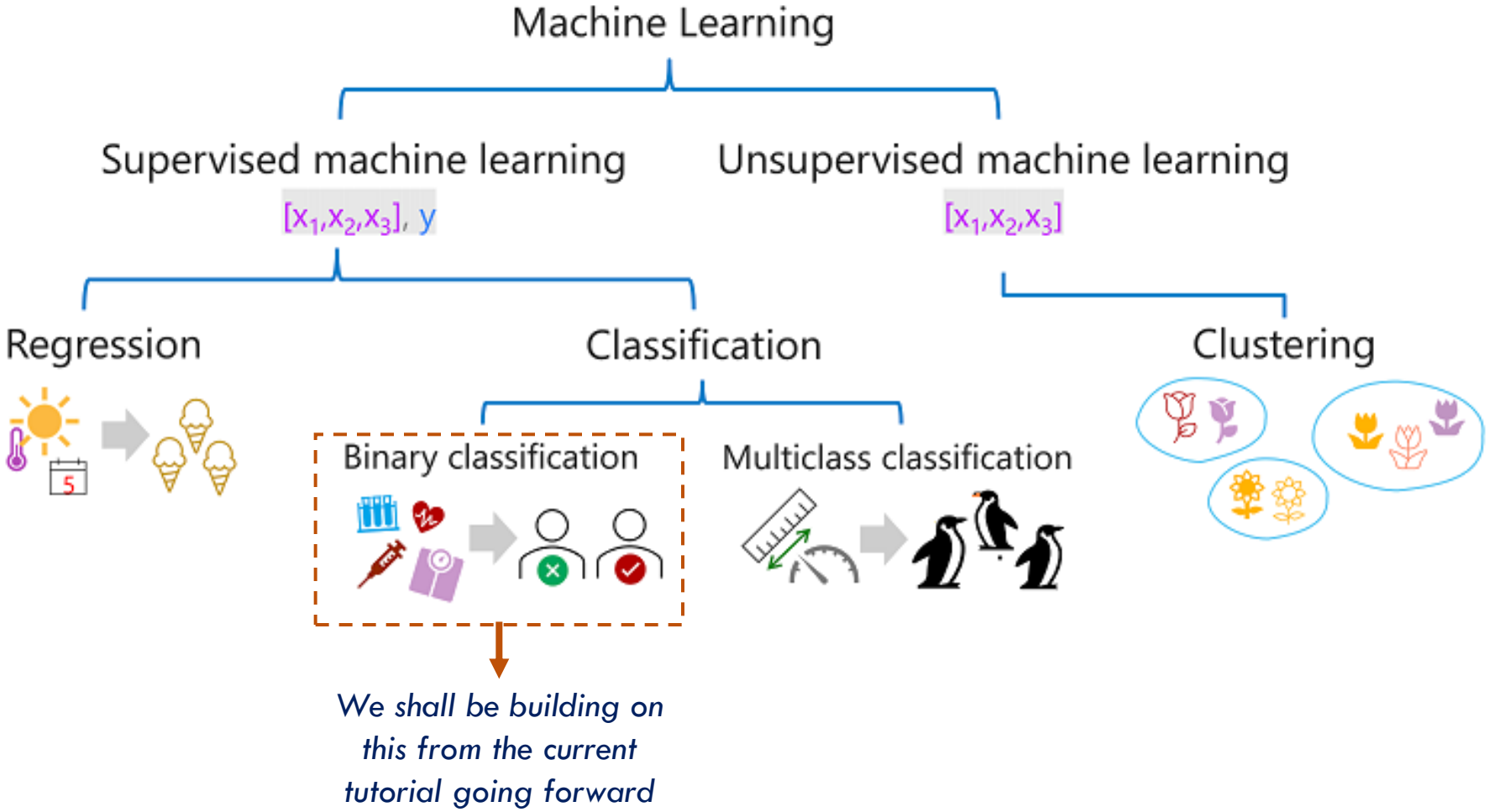
Also called *reinforcement learning from human feedback (RLHF)*, is a type of dynamic programming that trains algorithms using a system of reward and punishment, enabling an agent to learn best strategies

5. Semi-supervised Learning

Offers a combination between supervised and unsupervised learning. Semi-supervised learning algorithms are trained on a small labeled dataset and a large unlabeled dataset, with the labeled data guiding the learning process for the larger body of unlabeled data

Source: Microsoft Learn

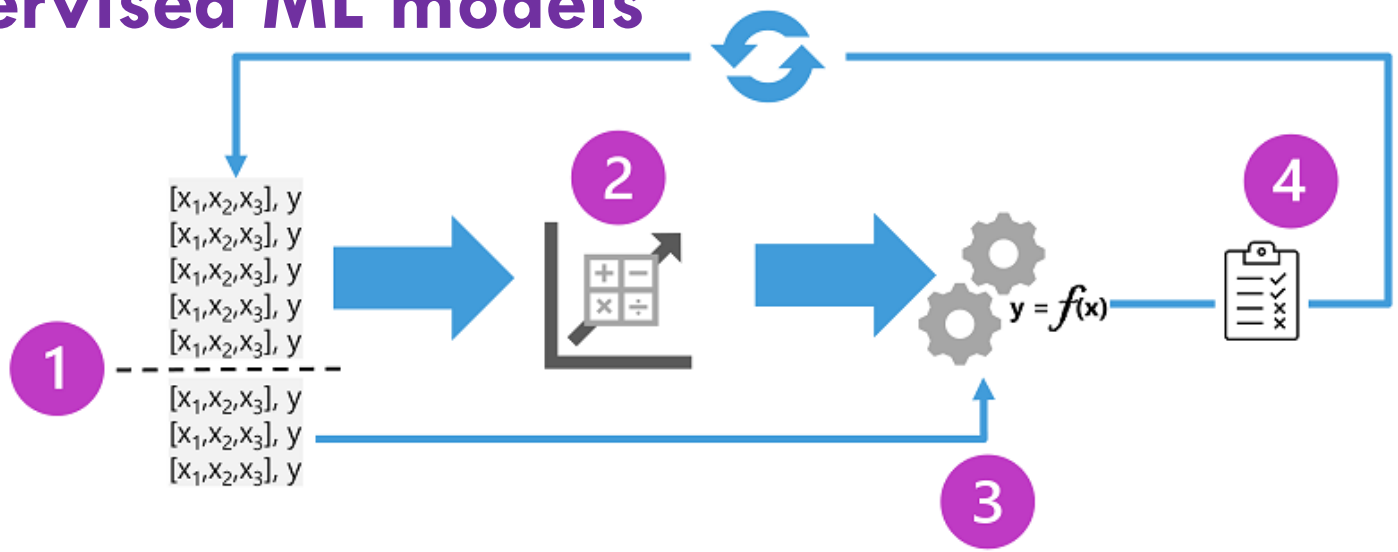
Visual summary of basic ML types



Excluding Self-supervised, Semi-supervised and RL since those are more advanced topics

Source: Microsoft Learn

Key elements of the training process for Supervised ML models



1. Split the data randomly into a training and test / validation set. The model will only be trained using the training set
2. Use an algorithm to fit the training data to the model. In the case of regression, a regression algorithm is used. For classification, a classification algorithm is used
3. Perform inference on the trained model using the held back validation data
4. Compare the known actual / true labels in the validation dataset to the labels the model predicted, in a process known as model evaluation, using evaluation metrics

Now, On to Scikit-Learn…



[https://scikit-learn.org/stable/]

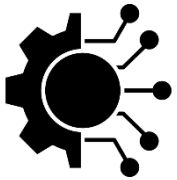
sklearn is an open-source gold-standard Machine Learning library for the python programming language

Why scikit-learn (sklearn)?

- ✓ It's built on top of numpy, matplotlib, scipy, and python – all of which we've been working with so far (except scipy, that we don't have to worry about)
- ✓ Integrates well with pandas and other frameworks
- ✓ Has built-in machine learning models and evaluation tools
- ✓ Beautifully designed API with easy-to-learn workflow
- ✓ Incredibly detailed, extensive, well-organized documentation with code examples all the way – (seriously though, sklearn has hands-down one of the best documentations of all ML frameworks out there)

⚠ **Just to reinforce the last point, try comparing sklearn documentation to other frameworks used for ML such as pytorch and LangChain**

sklearn key features…



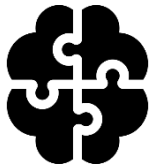
(a) Algorithms

Offers a broad range of established machine learning algorithms for both supervised and unsupervised learning: classification, regression, clustering, dimensionality reduction, ensemble methods, e.t.c



(b) Data preprocessing

Utilities for cleaning and preparing data, including handling missing values, scaling, normalization, and feature extraction



(c) Model selection and evaluation

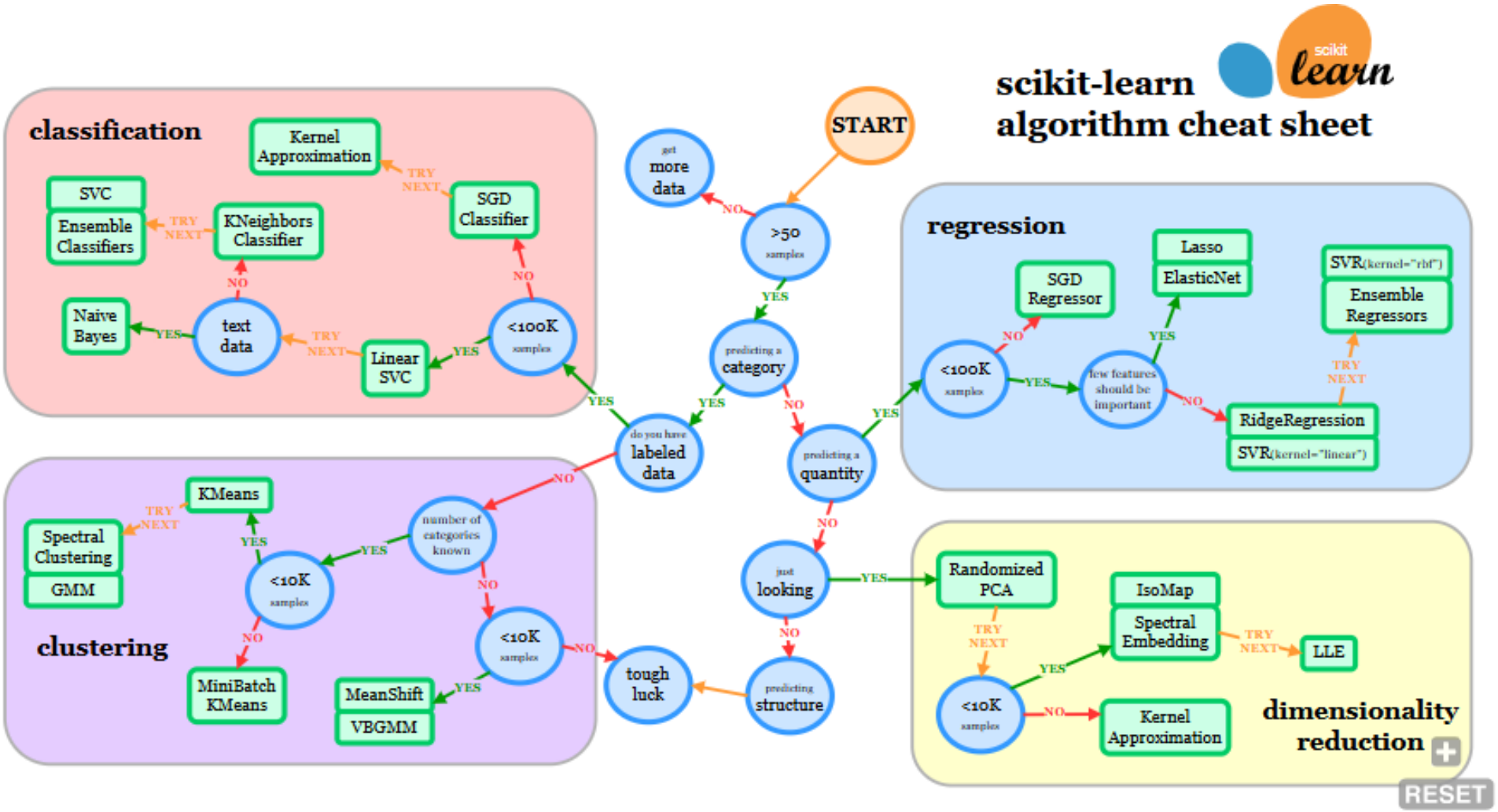
Tools for comparing, validating, and tuning models, such as cross-validation, grid search, and various performance metrics (e.g., accuracy, precision, mean squared error)



(d) Pipelines

The Pipeline utility allows users to chain multiple preprocessing and modeling steps into a single object, ensuring a streamlined, efficient, and reproducible workflow

Source: sklearn docs linked [here]



The sklearn documentation offers a guide to selecting the appropriate algorithm. Link attached above

Source: zerotomastery.io

A typical sklearn workflow…





Let's code it out!

The dataset for this coding tutorial has been provided
Also, I'll be using Google Colab for this tutorial.
Therefore, the file paths and output will definitely look different



Data Wrangling

ℹ The process of cleaning, and structuring, data into a usable format

New Jupyter Code Cell

```python
# imports
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

New Jupyter Code Cell

```python
# read in the dataset
students = pd.read_excel('/content/student_knn_data-gemini-3.xlsx', engine='openpyxl')

# display dataset summary info
students.info()
```

1. As always, the first thing we do is import necessary libraries
2. In the second code cell, we read in the dataset, which is in excel format, hence we use `pd.read_excel` function, then display summary using `.info()` function
3. NOTE: notice, for this dataset, NULL values exist

CONTINUED ▶

New Jupyter Code Cell

```
1 # check null values per column
2 students.isnull().sum()
```

1. **Here, we check the total missing / NULL (NaN) values that are present for each column in our dataset**

CONTINUED

New Jupyter Code Cell

```
1 # display rows for which ANY value is NULL (NaN)
2 # axis=1 specifies 'across the rows'
3 students[students.isnull().any(axis=1)]
```

1. Next, this line of code is used to filter the `students` DataFrame for rows where any value is missing / NULL / `NaN`
2. `students.isnull()` checks every cell in the `students` DataFrame and returns a DataFrame of the same shape, but with: `True` where the value is missing `(NaN)`; `False` where the value exists
3. The `DataFrame.any()` method returns `True` where at least one element within a DataFrame's specified *`axis`* is `True`. In this case, `.any(axis=1)` specifies that the operation be done across each row, of the resulting DataFrame and return a **Boolean Series**
4. Hence, `students.isnull().any(axis=1)` returns a Boolean Series of `True` & `False` values; `True` where the row has a missing value, and `False` where the row has all values present
5. Plugging `students.isnull().any(axis=1)` into the students DataFrame as illustrated in the code cell above, filters out the rows for which our condition is met

ⓘ This DataFrame filtering technique is known as Boolean Indexing. Read more on that [here]

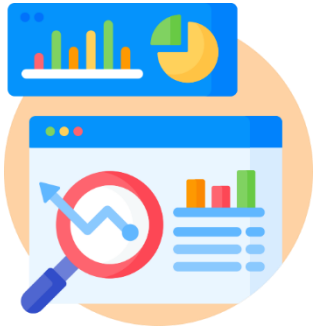
CONTINUED ▶

New Jupyter Code Cell

```
# drop rows with null values, drop the No. colum
students.drop(columns=['No.'], inplace=True) # drop column
students.dropna(inplace=True) # drop rows with null values

# get info on new DataFrame
students.info()
```

1. Next, we have a column, ‘No.’ that we shouldn’t include in our features since it wouldn’t be predictive of the label, ‘Final_Result’
2. So, we’ll drop the column ‘No.’ then we’ll drop all rows with any NULL values (both inplace)
3. To drop a column, we call `.drop()` on our students DataFrame
4. To drop rows with NULL values, we call `.dropna()` on our students DataFrame
5. Lastly, we call `.info()` on our DataFrame to get a summary of our new DataFrame. Notice, now, that the number of entries per column are balanced, meaning no NULL values.



Exploratory Data Analysis (EDA)

The critical initial process of analyzing, visualizing, and summarizing datasets to understand their structure, uncover patterns, detect anomalies, and test assumptions before building models

Why Data Wrangling & EDA?

- ✓ These steps reveal the extent of missing values, and allows us to handle them
- ✓ Helps detect outliers and decide how to handle them.
- ✓ Also, might inform the algorithm to select
- ✓ Help in feature selection and engineering
- ✓ Helps detect biases in data, features, and the label
- ✓ Helps check for categorical vs numeric features / column which informs data pre-processing steps



Data Wrangling and EDA go hand in hand. One could argue that we still performed EDA in the previous section

(a) Check the target / label column

New Jupyter Code Cell

```
# check unique count in target column
students['Final_Result'].value_counts()
```

New Jupyter Code Cell

```
# plot that on a bar plt
plt.figure(figsize=(5,3), layout='compressed')
# plot the Series above
students['Final_Result'].value_counts().plot.bar(color=['#AB274F', '#007FFF'],
                                                 rot=0)
# rot = 0 argument above ensures xaxis ticks (PASS, FAIL) appear horizontal

plt.title('Label Distribution') # set plot title
plt.ylabel('Count') # xaxis label

plt.show()
```

CONTINUED

New Jupyter Code Cell

```
# plot that on a bar plt
f, ax = plt.subplots(figsize=(5,3), layout='compressed')
# plot the Series above
students['Final_Result'].value_counts().plot.bar(ax=ax, color=['#AB274F', '#007FFF'])

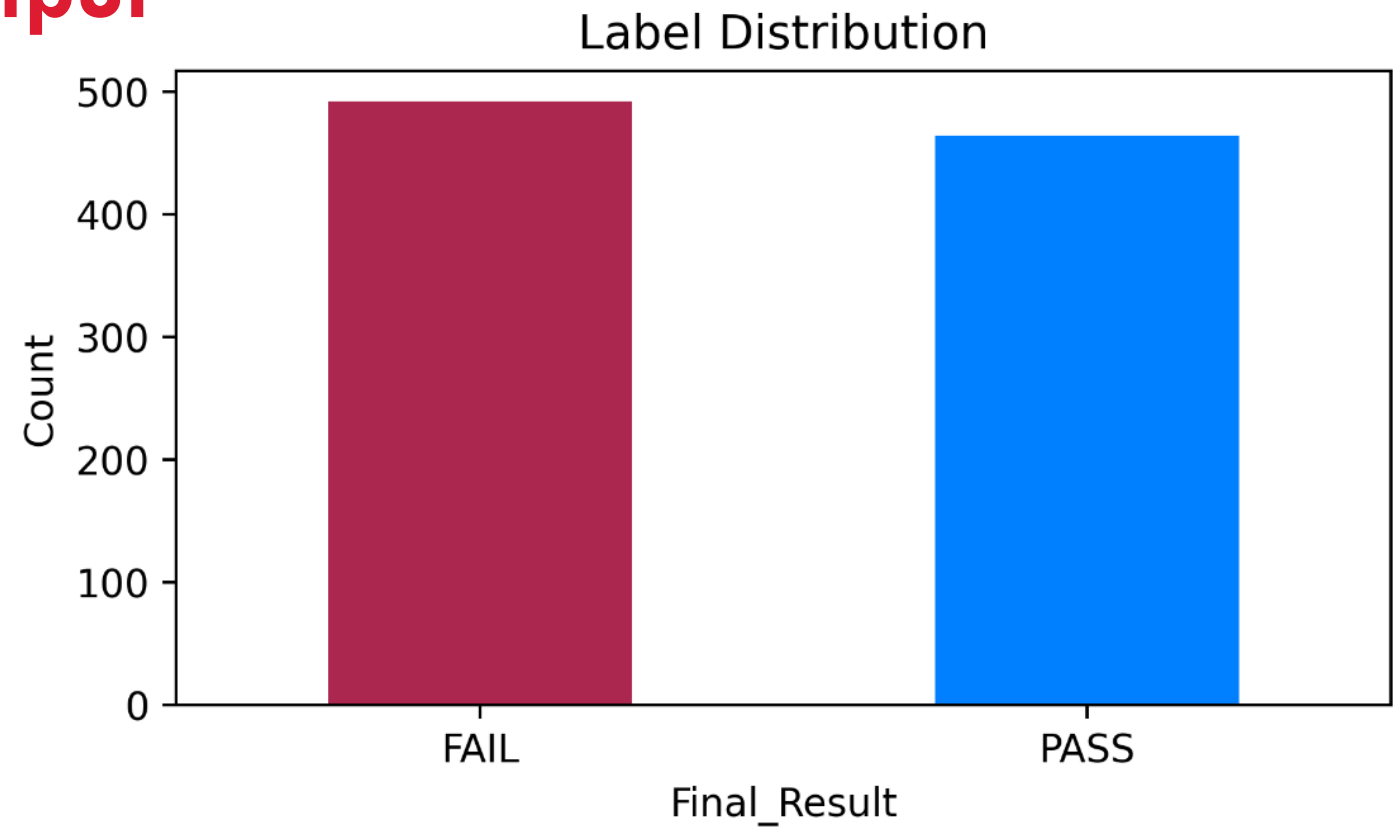
ax.set_title('Label Distribution') # set plot title
ax.set_ylabel('Count') # xaxis label
ax.xaxis.set_tick_params(labelrotation=0) # make xaxis ticks (PASS, FAIL) horizontal

plt.show()
```

1. In the previous slide, accessing the target column and calling `.value_counts` on it as such: `students['Final_Result'].value_counts()` displays the distribution of 'PASS' and 'FAIL' values.
2. Next, it makes much more sense to visualize that distribution in a Bar plot
3. Both plotting methods display the same exact plot, only that in the previous slide, I used the pyplot approach, and in the current slide, the `Axes` approach (check handout provided in the previous tutorial class)

CONTINUED ▶

Output



1. **The classes in the label do not appear to be severely imbalanced.**
2. **This is advantageous since we'll have a straightforward, reliable and less biased modelling process**
3. **If the classes were greatly imbalanced, we would have to perform resampling techniques (such as SMOTE) and rely on specialized classification metrics (such as recall, precision and f1) to measure model performance**

(b) Check association between the label column & other columns

The label and potential features in a dataset can either be:

(i) Numeric – representing a continuous number

(ii) Categorical – representing a group / category / class

- ✓ In our dataset, the label is categorical (PASS or FAIL) and the rest of the columns are numeric
- ✓ Hence, to visualize relationship between categorial and numeric variables, we use a boxplot. We shall use a boxplot to visualize distribution of the numerical column values across the label: PASS or FAIL
- ✓ Alternatively, if we were exploring relationship between numeric columns, we would have used a **correlation matrix** or **scatter plot**, or **both** – for good measure

*** For explanation on what a boxplot is and how to interpret one, watch the YouTube video linked [here] ***

New Jupyter Code Cell

```
# visualize relationship for one column (Hours_Studied)
f, ax = plt.subplots(figsize=(2.5,2.5), layout='compressed')
students.plot.box(by='Final_Result', column=['Hours_Studied'], ax=ax)
plt.show()
```

1. Here, we use the Axes approach to plot a boxplot that shows distribution of values in the 'Hours_Studied' column across the two label classes, FAIL and PASS (plotting was covered in the previous tutorial – check the slides)
2. Since we want to visualize the same relationship for the rest of the columns as well, we'll have to use a loop to achieve that

New Jupyter Code Cell

```python
# list of column names we want to plot
cols = ['Hours_Studied', 'Attendance_Percentage', 'Assignments_Completed',
        'Past_Exam_Score']
```

New Jupyter Code Cell

```python
# define figure and subplots
f, axs = plt.subplots(nrows=1, ncols=len(cols), figsize=(8, 2.5), layout='compressed')

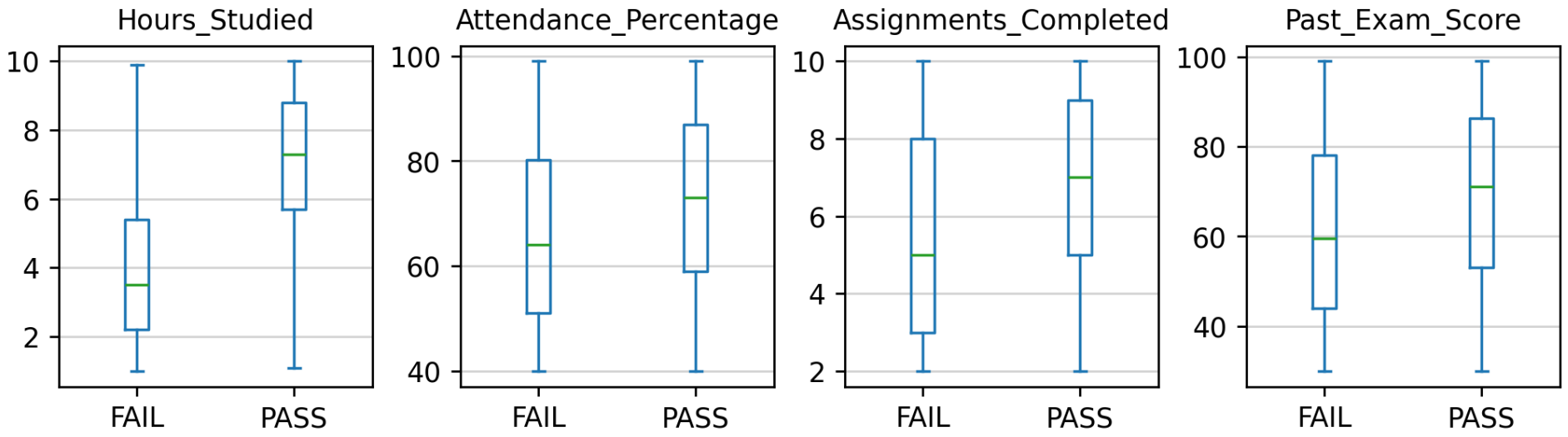
# use a for loop to plot in all Axes in axs
for i, ax, in enumerate(axs):
  students.plot.box(by='Final_Result', column=cols[i], ax=ax)
  ax.set_title(cols[i], fontsize=10)
  ax.grid(axis='y', color='lightgray')

# suptitle
f.suptitle('Correlation Between Final_Result and other Columns', weight='black')
plt.show()
```

1. enumerate is just used to add a counter as we loop, and the values returned by this counter are used to access values in `cols`, index-wise

CONTINUED ▶

Output





The box plots reveal a notable variation in how 'Final_Result' relates to the other numeric columns. This indicates a difference in the distribution of values within each numeric column across the two label classes, PASS and FAIL, suggesting that these numeric columns can effectively predict the students' 'Final_Result'.

Therefore, all numeric columns ['Hours_Studied', 'Attendance_Percentage', 'Assignments_Completed', 'Past_Exam_Score'] will be regarded as important features and utilized for training our model.

Hyper-links to docs are included below

(c) Encode the label

Encoding is the process of converting non-numerical, categorical values (such as labels like ‘red’, ‘green’, ‘blue’ or ‘male’, ‘female’, ‘PASS’ or ‘FAIL’) into a numerical format that machine learning algorithms can understand and process

- ✓ It is necessary because most machine learning algorithms are designed to work exclusively with numerical data and cannot process text or categorical strings
- ✓ In our case, we shall encode our label column, ‘`Final_Result`’ so that all values of FAIL become `0`, and all values of PASS become `1`
- ✓ There’s several approaches to achieving this, but to keep it simple, we’ll go over 4 approaches as discussed in the subsequent slides
- ✓ These approaches are: `Series.replace`, using `LabelEncoder`, `Series.map` and `Series.apply` to achieve the same result

ⓘ Recall, that selecting a single column from a DataFrame returns a Series object

(i) APPROACH 1: Series.replace()

Links to docs included

New Jupyter Code Cell

```
# --- APPROACH 1: Series.replace() ----
# overwrite values in label column with results from transformed values
students['Final_Result'] = students['Final_Result']\
      .replace({'FAIL': '0', 'PASS': '1'})\
      .astype(int)

# visualize first 5 rows
students.head(5)
```

1. In the first approach, we use `.replace()` method and a provide a `dict` mapper
2. In our case, we use a `dict` as a mapper, where the keys specify the targets to replace, and values, the substitutes. Note that we specify `'0'` and `'1'` as strings to avoid errors when substituting between different data types
3. We then call `.astype(int)` to convert the `'0'` and `'1'` string values into integers
4. Finally, we overwrite the original column in the DataFrame with encoded values by assigning back to the column

(ii) APPROACH 2: LabelEncoder

Link to docs included

New Jupyter Code Cell

```python
# --- APPROACH 2: LabelEncoder ----
from sklearn.preprocessing import LabelEncoder

# initialize an object of LabelEncoder
le = LabelEncoder()

# overwrite values in label column with results from transformed values
students['Final_Result'] = le.fit_transform(students['Final_Result'])

# visualize first 5 rows
students.head(5)
```

1. Here, we use the sklearn LabelEncoder class to convert categorical values in our label column into numeric labels automatically. Unique values are sorted alphabetically, then assigned integers starting with `0` to `n-1`
2. Finally, we overwrite the original column in the DataFrame with encoded values by assigning back to the column

(iii) APPROACH 3: Series.apply()

Link to docs included

New Jupyter Code Cell

```python
# --- APPROACH 3: Series.apply() ----
def encoder_func(x: str) -> int:
  return 1 if x == 'PASS' else 0

# overwrite values in label column with results from transformed values
students['Final_Result'] = students['Final_Result']\
                              .apply(encoder_func)
                            # .apply(lambda x: 1 if x=='PASS' else 0)

# display first 5 rows
students.head(5)
```

1. Series.apply() is used to invoke a function on each value of a Series object
2. In our case, we define a function encoder_func that returns 1 if the particular value in the Series is PASS, otherwise 0. Note that when specifying the function inside .apply() we provide the function name only
3. Alternatively, we can specify an anonymous / lambda function that implements the same logic as our encoder_func as shown in the comment

ⓘ Read on python lambda functions [here] or watch the insightful video linked [here]

(iv) APPROACH 4: Series.map()

Link to docs included

New Jupyter Code Cell

```
# --- APPROACH 4: Series.map() ----
# overwrite values in label column with results from transformed values
students['Final_Result'] = students['Final_Result']\
                                .map({'PASS':1, 'FAIL':0}, na_action='ignore')

# visualize first 5 rows
students.head(5)
```

1. Series.map() is used for substituting each value in a Series with another value, that may be derived from a function, a dict or a Series
2. In our case, we use a dict as a mapper, where the keys specify the targets to replace, and values, the substitutes
3. Finally, we replace the original values in the DataFrame with transformed values by assigning back to the column

In our current very simple project, this is the simplest and most straight-forward approach

(d) Specify features, X, and label, y

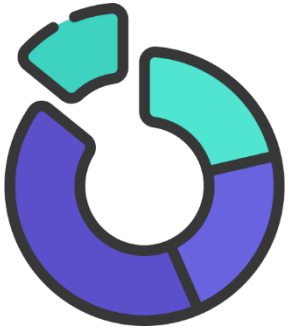
New Jupyter Code Cell

```
# X - features, y - label or target
X = students[['Hours_Studied', 'Attendance_Percentage',
              'Assignments_Completed', 'Past_Exam_Score']]

y = students['Final_Result']
```

1. Recall, we established that all numeric columns will be considered useful features
2. Hence, all column aside the label are specified as features, and assigned to a variable, X
3. The 'Final_Result' column is specified as the label / target and assigned to a variable, y

sklearn from here henceforth; Hyperlinks (underlined blue text) to relevant docs included



Splitting Data into Test & Train sets

ℹ Usually, we split training data (randomly) to create a dataset with which to train the model while holding back a subset of the data that we'll use to validate the trained model.

To split our data, we shall use `train_test_split` from sklearn

New Jupyter Code Cell

```python
from sklearn.model_selection import train_test_split

# split our features, X, and label, y, into test and train sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42)
```

New Jupyter Code Cell

```python
# check size of training set
X_train.shape, y_train.shape
```

New Jupyter Code Cell

```python
# check size of test set
X_test.shape, y_test.shape
```

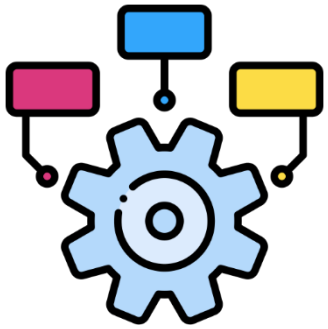
CONTINUED

Link to docs included

Explanation

1. In the first code call, we import `train_test_split`, then specify a number of arguments
2. We pass `X`, as the features and `y` as corresponding labels / targets
3. We also specify that we want the test set to be 20% of the dataset passed, by passing `test_size=0.2`
4. Also, to control the shuffling applied to the dataset, we assign `random_state` parameter so we can always have reproducible outputs across multiple function calls
5. `train_test_split` returns 4 values that are captured as shown
6. In the other two code cells, we print out the shape of the train and test sets respectively – to get an idea of how the dataset was split

For in-depth explanation of train_test_split, read the docs linked [here]



Data Pre-processing

ℹ The process of transforming, and data into a structure that can be used for model training

Let's take a detour…

Recall, that features can either be:

(i) Numeric – representing a continuous number

(ii) Categorical – representing a group / category / class

- ✓ Among other goals, data preprocessing aims to scale numeric features and encode categorical features
- ✓ Encoding entails **transforming non-numeric categorical values such as 'summer', 'winter', 'spring', 'autumn' into numeric representations**. This is important because most models expect purely numeric input and perform mathematical operations on numbers as opposed to strings
- ✓ Scaling involves **transforming the range / distribution of numerical features into a consistent common scale**, for instance, [0 to 1], or [-3 to +3]. Scaling is necessary because numeric features have varying ranges. For instance, `age` might range from 0 – 100, and `salary` might range from 0 – 1,000,000. If not scaled to a common range, `salary` might dominate, making `age` irrelevant as a predictive feature

Link to docs: Pipeline

Pipeline...

Assume we have a number of feature preprocessing steps, such as scaling, encoding, imputing, e.t.c. We could choose to implement these, one at a time, by creating class objects and transforming + overwriting original data



Note:

`sklearn` simplifies this process by providing a single entity, `Pipeline`, that allows us to chain multiple data pre-processing steps, and an optional final Machine Learning algorithm (or *predictor*, - as sklearn calls them) into a single unified object. The benefit of this approach is that:

1. The pipeline bundles preprocessing & modelling into one object, ensuring that the exact same steps are applied during training and inference
2. When training, it *saves* statistics, and rules (like means) learned from training data
3. At inferencing, it applies those *saved* rules to, ensuring inference data is transformed in the exact way as training data

ⓘ We shall cover bundling preprocessing steps & estimators in the next tutorial

Hyper-links to docs are included below

sklearn data pre-processing tools…

1. `StandardScaler` — Performs standardization of numerical features so that they have a mean of 0 and standard deviation of 1, and this happens independently on each feature

2. `MinMaxScaler` — Transforms numerical features by scaling them to a given range; by default (0 - 1)

3. `SimpleImputer` — A simple way for completing missing values using simple strategies (statistics such as median, mean, most_frequent)

4. `ColumnTransformer` — Applies transformers (pre-processing operation) to specific columns of an array or DataFrame

5. `OneHotEncoder` — Encodes categorical features (Turns categorical features into numeric format, usually an array of 0 and 1)

ℹ These are just a few of the most common and simplest.



Let's code it out!

To understand Pipeline and pre-processing tools, let's code it!
The two datasets for this section are provided

New Jupyter Code Cell

```
# imports
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
```

New Jupyter Code Cell

```
# read in csv with Null values
games = pd.read_csv('/content/game_stats.csv')
sales = pd.read_csv('/content/sales.csv')
```

1. In the first cell, we've imported all the tools we need for pre-processing
2. In the second cell, we read in the datasets as DataFrames and hold them in variables, `games` & `sales`, nothing we haven't done before

New Jupyter Code Cell

```
# display sales
# purely numeric
sales
```

New Jupyter Code Cell

```
# display games
# both categorical & numeric
games
```

1. From the cell outputs, we can see that `games` contains both numeric and categorical columns while `sales` contains purely numeric columns
2. The datasets have been specifically kept small for ease of visualization

New Jupyter Code Cell

```
# pre-process sales dataset
sales_pipeline = Pipeline(steps=[
    # (name of step, estimator / transformer)
    ('imputer', SimpleImputer(strategy='median')), # step 1
    ('scaler', StandardScaler())                   # step 2
])

# visualize the pipeline object as a diagram
sales_pipeline
```

New Jupyter Code Cell

```
# applying those preprocessing step to sales
sales_pipeline.fit_transform(sales)
```

1. **Here, we initialize an `sklearn Pipeline` and specify the steps through which data flows as a tuple:** `(name of step, transformer / estimator object)`
2. **The first step is a `SimpleImputer` object that will fill any NULL values with the median of the feature / column values**
3. **The second step is a `StandardScaler` object, that will scale the features**
4. **Calling `.fit_transform()` on the Pipeline object will fit all the transformer steps sequentially, then transform the data sequentially**

New Jupyter Code Cell

```
# plug into a dataframe for easier visualization
pd.DataFrame(sales_pipeline.fit_transform(sales))
```

1. This step is for ease of readability
2. Note how the data has been transformed. Values that were in the thousands have been condensed into small decimals
3. Also, note how positions with NULL values, previously identified as `NaN` in the `sales` DataFrame have been filled with values

New Jupyter Code Cell

```python
# transform games dataset
num_transformer = Pipeline(steps=[ # for numerical columns
    # (name of step, estimator / transformer)
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

cat_transformer = Pipeline(steps=[ # for categorical columns
    # (name of step, estimator / transformer)
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('ohe', OneHotEncoder(handle_unknown='ignore'))
])

# combine both transformers
preprocessor = ColumnTransformer(transformers=[
  # (name of step, transformer, [target columns])
    ('cat', cat_transformer, [0,1]),
    ('num', num_transformer, [2,3,4])
])

# display preprocessor as a diagram:
preprocessor
```

CONTINUED



Explanation

1. The `games` dataset has categorical and numerical columns, hence we can't transform the entire dataset the same way.
2. We have to account for both numerical columns by scaling them, and categorical columns; by encoding them
3. We wrap transformers for numerical and categorical features as distinct `Pipeline` with distinct transformation steps
4. The numerical transformer `num_transformer` will fill any NULL values with the feature's `median` value, and then standardize the numeric features (sequentially)
5. The categorical transformer `cat_transformer` will fill any NULL values with the `most_frequent` value in that specific feature, then encode the categorical features using `OneHotEncoding` (again, sequentially)
6. Finally, these two transformation steps are bundled into a `ColumnTransformer` object that specifies what columns are transformed how
7. In this case, `cat_transformer` is applied to columns at index `[0, 1]` (which are categorical columns in the `games` dataset
8. Also, `num_transformer` is applied to columns at index `[2, 3, 4]` (which are numerical columns in the `games` dataset)

New Jupyter Code Cell

```
# applying those preprocessing step to games
preprocessor.fit_transform(games)
```

New Jupyter Code Cell

```
# plug into a dataframe for easier visualization
pd.DataFrame(preprocessor.fit_transform(games))
```

1. This step is for ease of readability
2. Note how the data has been transformed. Numerical columns (the last 3 columns) have been scaled into decimals
3. The categorical columns have been scaled in a One-Hot fashion, hence the 9 new columns (for every unique value in the categorical columns, `OneHotEncoder` creates a corresponding new column, then assigns `1` where the value exists, or `0` where it doesn't exist – you don't have to worry about this though)
4. Also, note how positions with NULL values, previously identified as `NaN` in the `games` DataFrame have been filled with values

What Next?



Now that we have the basics on creating a sklearn Pipeline and specifying data transformation steps, in the next tutorial lesson, we'll add a scikit-learn predictor (ML algorithm) into the Pipeline. Then we'll train, evaluate it, and make inferences!

⚠ We shall be training a KNN classifier algorithm. Watch the YouTube video linked [here] for insight on what KNN entails

Official python documentation linked [here]



References

Here's a list of recommended references for python, pandas, numpy, scikit-learn, matplotlib, seaborn and all things data science:

- Official pandas documentation linked [here]
- Official numpy documentation linked [here]
- Official matplotlib documentation linked [here]
- Official scikit-learn documentation linked [here]
- Official seaborn documentation linked [here]
- RealPython – website linked [here]
- Medium – website linked [here]



Up Next:

1. Bundling data pre-processing + model in a single pipeline
2. Model training & inferencing
3. Model evaluation